

Data Mining Project

Group 12

Davide Borghini Andrea Marino
d.borghini3@studenti.unipi.it a.marino47@studenti.unipi.it

Yuri Ermes Negri
y.negri@studenti.unipi.it

January 5, 2025

Abstract

Report of the Data Mining exam's project, a.y. 2024/2025. The project consists in the full implementation of a Data Mining pipeline on the data given by the professors.

The data is about professional cyclist races, and consists of two datasets, as described in the handed out guidelines document.

This document is subdivided into sections that follow the five tasks in which the project is divided. In each section the activity performed for the corresponding task will be explained, together with key insights on the discoveries and achieved results, a description of the followed methodology, a discussion of strenghts and weaknesses of our approach, and with a discussion on further possible improvements.

Our code has been delivered on GitHub according to the instructions, on a private repository. However, our full work is available on the original GitHub repository.

Contents

1	Introduction	3
2	Data Understanding and Data Transformation	3
2.1	Preliminary data exploration	3
2.2	Features exploration and analysis	6
2.3	Closing remarks	11
3	Clustering	11
3.1	Prototype Clustering	11
3.2	Density-based Clustering	13
3.3	Hierarchical Clustering	15
3.4	Observations and comparisons	17

4	Classification and Explainability	17
4.1	Validation strategy	18
4.2	Metrics	18
4.3	Decision Tree	19
4.4	Decision Forests	19
4.5	Neural Networks	21
4.6	Performance on Test Set	23
5	Conlusions	25

1 Introduction

Cyclism has proved to be quite a complex matter. In this project, we used two datasets to delve into its complexity: one for the cyclists and the other for the races they participated in.

The project was divided into five tasks: Data Understanding, Data Transformation, Clustering, Classification, Explainability. We decided to merge the first two and to merge to last two into two sections, because the work we did in each was deeply connected, and we frequently revisited the work for those pairs of tasks jointly, in an iterative fashion.

In the following sections, there can be seen our dataset analysis, alongside the feature we engineered and the results of Prototype, Density and Hierarchical based clustering algorithms on them. Also, in the last section, the work we did for the Classification and Explainability part is described, the performances over the Test Set of our chosen model is reported.

2 Data Understanding and Data Transformation

Originally two distinct tasks, we merge them into one section because in practice the two aspects of understanding and transforming the data went hand in hand, thus it has been a cyclic process rather than a sequence of steps. Together, these parts are preponderant in our project.

In our code, this part is spread over the following documents: `data_exploration`, `data_understanding_transformation`, `delta_final`, and also `proccyclingstats`.

2.1 Preliminary data exploration

The first thing we did was to study the cyclists data and the races data independently, focusing on the following aspects:

- **Duplicated data**, i.e. check wheter there were duplicated rows, or something more subtle. All in all, check if there were duplicated values where there shouldn't have been.
- **Missing data**, i.e. check how much data was missing for each feature, but also how much was missing overall in the dataset, more in general.
- **Consistency checks**, i.e. understand whether the values in the datasets made sense according to common sense criteria. In case they didn't, we tried to understand the underlying phenomenon that caused these alleged mistakes, and took action consequently, sometimes excluding the data, sometimes fixing it.

2.1.1 Web scraping

In general, the problems with missing data and inconsistencies led us to try to recover some of it from ProCyclingStats [1] - which is an authoritative source - by using some web scraping tools [2]. This is performed on the `proccyclingstats` notebook.

The Python package made the task easy, but extra care had to be taken because its classes' methods were very sensible to missing and duplicated data. Managing the library's brittleness helped us in extracting knowledge from the data even more. In general it turned out that the data on ProCyclingStats had many problems that also the original data had, thus the data quality was good overall. Almost as good as the best publicly available source.

In the scraping, the original data was given precedence. That is, if a value was present in the dataset, we didn't scrape the web. We did so to minimize the calls to the scraping methods (which take time), to be as close as possible to the original data, and because a preliminary study showed that the latter was overall consistent with ProCyclingStats.

Furthermore, on the website many stages listed many more cyclists, but some did not finish, or finished outside the time limit, or were disqualified. These conditions are explicitly marked on the website. The so called "dropouts" weren't present in the original dataset, thus we didn't introduce them; nor we recovered races that are missing in the dataset.

For the cyclists, there have been no noticeable changes, but for the races, the most noticeable changes are the introduction of the `stage_type` feature - denoting whether the stage¹ is an individual time trials (ITTs), i.e. a stage which is ran with slightly different rules than the usual road race (RR) stages - and the overwriting of the `points` and `uci_points` values with scraped data.

We decided to do the latter change because in the original dataset all the cyclists in a given race are given the same amount of (UCI) points, namely those of the first cyclist. The range of the two features was so small that they behaved like categorical variables. Furthermore, the scraped values are tied to the cyclist in the race, which is a huge gain.

2.1.2 Cyclists

There were no duplicated rows, although there were homonymous cyclists. Very few cyclists missed the `birth_year` feature, and it wasn't possible to retrieve this value from the `cyclist_age` feature of the races dataset, even after the scraping.

Out of 6134 cyclists, 2991 missed the `height` feature, 3054 out of 6134 cyclists missed the `weight` feature, and almost all of those who missed either missed also the other. This made more difficult the following tasks, but despite this we decided not to impute these values, as they wouldn't be true, thus they'd be misleading.

2.1.3 Races

There were no duplicated rows, but there were multiple cases of cyclists appearing twice inside the same race, with different `position` and `delta` values. Often these problems were also present on ProCyclingStats. Upon further investigation (and questioning domain experts), we understood that sometimes it's the data gathering process on behalf of ProCyclingStats that has problems, often due to ex-post penalizations, or other niche phenomena that aren't captured by a bare leaderboard (illness, injuries, ...).

¹Single day races have value RR (Road Race) for this feature

We kept just the first of such "copies".

We also noticed that some competitions had multiple `name` values associated to them, but the `_url` proved to be a reliable identifier. Speaking of which, we used these values to pinpoint which competitions are staged and which are single day races. This information has been encoded in the `is_staged` feature.

We saw that some competitions aren't completely present in the dataset, and in general not all competitions are equally represented. Six of the 27 competitions had no missing editions nor stages, for the others something was missing. Some competitions missed one edition completely (for example the Ronde Van Vlaanderen of 1998 is missing) and some competition missed few stages of few editions, as well as all stages of one edition (one example of this is the Tour de France). Some missed more, and some less. The presence is very good overall, and it can be checked in the `data_exploration` notebook. Keep in mind that the assessment is based on the data handed to us, not on comparison with external sources. Thus we can't (e.g.) say if there had been editions prior to those present in the dataset (for some competitions like the Tour de France there are for sure),

Another interesting study, not directly related to any feature in particular, is that of attendance to the races. The participants in the various stages of the 27 different competitions in the dataset have been summed separately for each year (i.e. considering the editions independently), and the average number of participants in each edition of each competition could be computed. The result is visible in figure 6a on page 24.

Besides the comparisons among competitions, two main conclusions can be reached. Firstly, there has been a sudden surge in the number of participants for many competitions around 1996-2000. Secondly, besides the aforementioned surge in participation, the average number of participants varies a lot, but it's around the same number (150 circa) for all competitions, with variations in the tens range.

All this information is precious for classification, because it reveals that there's been a shift in the balancement of the classes in the labels' distribution, if we consider it in the temporal dimension.

The main study that was driven by checking the consistency of the values has been that of the `delta` feature, which is the time difference (in seconds) between the current cyclist and the first one. The entire `delta_final.ipynb` notebook has been devoted to this. The three criteria for assessing the consistency are non-negativity and monotonicity, with special attention towards the zero values.

It's important to say that we kept the `delta` values of the original dataset, but we compared them with those in ProCyclingStats.

The key results of this investigation are:

- It's not wrong nor unusual to give value 0 for delta to many cyclists. This is because when they arrive in groups (bunch sprint) the same time is recorded for all of them
- Sometimes the `delta` values are not (weakly) monotonically increasing as one would expect, and often this problem is also present on ProCyclingStats. Often the cause of this are the repeated cyclists already discussed

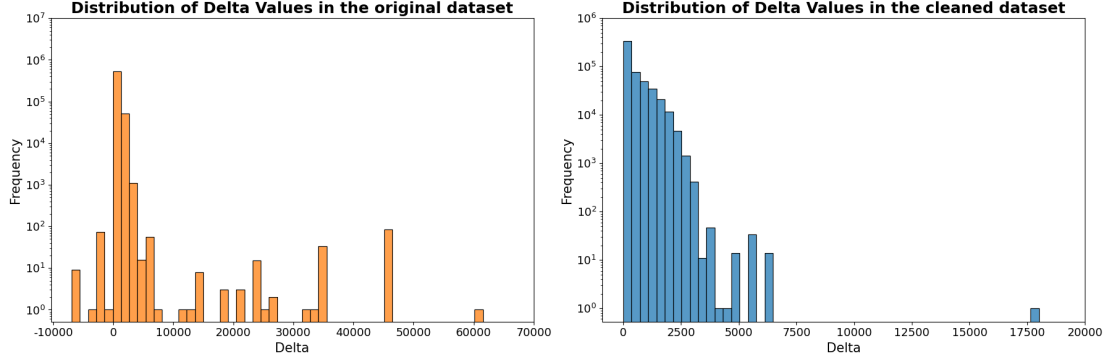


Figure 1: *Distribution of the `delta` values in the original (left) and cleaned (right) datasets*

- Few races had negative values for the feature, and this problem is present on ProCyclingStats as well
- There are huge delta values, and removing duplicated cyclists sometimes addresses this issue as well
- The second numerical value in the `date` column is not the start time of the race, nor is noise, but rather is the time it took the cyclist to complete the race

In the notebook, the various exceptions are analyzed and discussed one by one. After the analysis we decided to clean the dataset by keeping only the races that had consistent deltas (i.e. nonnegative and monotonically increasing), not before trying to fix some problematic races by removing the duplicated cyclists.

Before this cleaning, the dataset had 589865 rows. After the cleaning (including removing the duplicates), the dataset had 536361 rows. Thus, the newer version of the dataset had 90.93% of the number of samples of the original one. Figure 1 shows the situation before and after the process. The improvement is evident, it suffices to look at the range on the x axis, which is much more restricted.

The main advantage of our approach - compared to (e.g.) cutting away the extreme values of the delta - is that it's based on clear and reasonable principles, and is based on an investigation of the cause of the problems. Thus made it possible to solve them, and really taught us about the phenomenon behind the numerical values.

2.2 Features exploration and analysis

We ran an in depth exploration of the features; both those present in the two original datasets and those inferred by us. For these comparisons and assessments we focused mostly on the data obtained after the scraping phase. As said, it has been an iterative process: once we realized that the data quality wasn't satisfactory we decided to do data augmentation, and we committed to that decision. Besides, as discussed, the differences between original and scraped data were not that significant.

Before going over the engineered feature, let's list the features that we decided to discard. We discarded `is_gravel` and `is_cobbled`, because they're columns full of `False` values; `average_temperature`, because it missed approximately 94.93% of the values; and the `cyclist_team`. The latter was scraped, as there were differences between ProCyclingStats and the original dataset, but hasn't been used because it had too many distinct values (91 unique values in the original dataset, 3311 unique values in the scraped dataset), and because by speaking with a domain expert we understood that teams often change names even when they remain the same, unlike e.g. racing teams in Moto GP. Team play is an important part of professional cycling, but we judged that capturing phenomena at this level in further tasks would be too far fetched.

The engineered features are described in table 1.

Table 1: *Engineered features with their names, description, and dataset to which they belong*

Dataset	Feature Name	Description
Race	<code>stage_type</code>	Refer to sections 2.1.1 and 2.1.3.
	<code>is_staged</code>	
	<code>race_country</code>	Country of the competition, retrieved manually
	<code>steepness</code>	Vertical distance traveled divided by race length, expressed as a percentage
Race (cyclist-based)	<code>time</code>	Time taken by the cyclist to finish the race, extracted from the <code>date</code> column
	<code>average_speed</code>	Race length divided by the time taken by the cyclist
	<code>stamina_index</code>	Endurance indicator based on normalized race attributes, average speed, and position
Cyclist	<code>bmi</code>	BMI of the cyclist (weight divided by height squared)
	<code>race_count</code>	Number of competitions the cyclist participated in (stages of the same competition count as one)
	<code>total_points</code>	Total points earned by the cyclist, derived from the (scraped) <code>points</code> column
	<code>victories_by_points</code>	Number of competitions won by points (most points across all stages)
	<code>average_points_per_race</code>	Average points earned by the cyclist per (participated) race
	<code>average_position</code>	Average position achieved by the cyclist across all (participated) races

About the victories by points, keep in mind that for many competition that's different from actually winning (e.g. to win the *maillot jaune* in the Tour de France).

We ran an exhaustive analysis of the features, both the original ones and the engineered ones, often running pairwise, or even multiple-ways, comparisons among them. For brevity's sake, we just report a few significant results. The reader is redirected to the

Notebooks for more plots, further analysis and comments.

2.2.1 The `average_speed` feature

By checking the frequency count of this feature's value in different ranges with histograms (available in the `data_understanding_transformation` notebook), we discovered that the bulk of the mass of the distribution corresponded to speeds between 5 and 22 m/s, but there were many outliers. An in-detail analysis of the outliers was performed (again by comparing the data with ProCyclingStats to try to understand what happened and if it was fixable), and it turned out that often the races problematic with respect to this feature were also problematic with respect to the `delta` feature. However, after the investigation, we decided to further clean the dataset by keeping the races with speeds between 3 and 60 m/s. Of course we tried to recover some wrong values from ProCyclingStats, and some problematic races that had speeds between 22 and 60m/s have been individually removed once identified and understood the reason for such absurd values. The cleaning based on the `delta` consistency check did the rest, thus reducing the effective range of the feature's values even more.

We compared the `average_speed` feature with the `is_staged` feature and nothing significant emerged: the two types of races are ran at similar speeds. We compared the feature also with the `profile` categories, and according to a one-way ANOVA test the means of the `average_speeds` are different across the 5 profiles.

We also compared this feature with the `length` feature, despite the breadth of the analysis no key insight emerged, thus we won't stress this analysis.

2.2.2 The `length` feature

We discovered that the distribution of the `length` values decomposes really nicely when considering also the `profile` value. The "correct" way to look at this distribution is as the marginalization of the following distributions:

$$\mathbb{P}(\text{length}) = \mathbb{P}(\text{length}, \text{stage_type} = \text{ITT}) + \mathbb{P}(\text{length}, \neg \text{prologue}) + \mathbb{P}(\text{length}, \text{prologue})$$

Where $\neg \text{prologue}$ denotes that a race is not a "prologue" stage.² Prologue stages are all extremely short, and ran before the first Stage.

Figure 2 depicts the situation. The differences are clearly visible by eye. The point-biserial correlation between the length values and the `stage_type` binary distribution is 0.665, which increases to 0.774 when the prologue races are excluded. This shows that there's a clear distinction between the lengths of the ITT and RR races.

We ran a χ^2 test to assess if the observed values of RR lengths³ followed a normal

²There is only one race that is ITT and prologue, thus it's been left out of the analysis for simplicity (is included in the ITT races).

³Binning has a tremendous impact on the test, because the observed values and their frequencies change. The values were binned into 50 bins, but tried also with different number of bins, and also no bins at all.

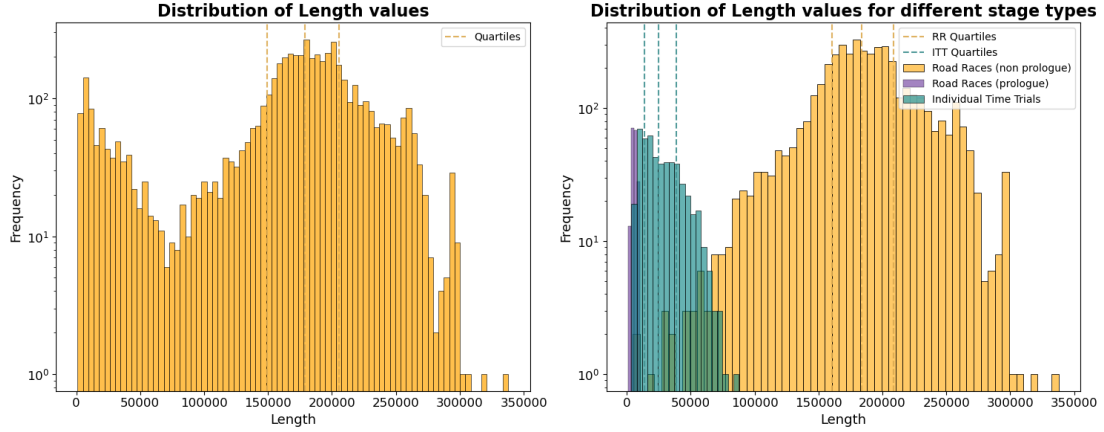


Figure 2: *Length distribution when marginalizing (left) and when not marginalizing the joint length-stage_type distribution (right).*

distribution, and the answer was negative.

2.2.3 The `startlist_quality` feature

The quality of the startlist has very precise rules for its computation, and on ProCyclingStats the `startlist_quality` of a race is broken down, thus in theory the scoring of each cyclist for each race is available. However, the scraping tools don't allow to directly access this precious data.

We saw that the cleaning steps outlined earlier (based on `delta` and on `average_speed`) don't alter this feature's distribution, which is great news, especially for the classification task.

A natural question that arose was whether the startlist quality had changed over the years. We found a slight positive correlation (0.214, using the `spearman` method) between the `date` and `startlist_quality` feature, and we grouped the latter feature's values into decades to compare across different time period. Simply visualizing the data seems to suggest some differences, but they haven't been validated with statistical hypothesis testing. Moreover, grouping into decades is rather arbitrary, a possible improvement would be to ask better year subdivisions to a domain expert, as maybe there have been "eras" of cyclism that we're unaware of.

2.2.4 The `total_points` feature

We compared the `total_points` values with the average position of the cyclists, and the number of attended races. By comparing the scatterplots in figure 6b on page 24 the following conclusions can be drawn:

- The overall majority of the cyclists haven't accumulated any points in their career, and many of them have a high value for `average_position`.

- There's a big lump of datapoints corresponding to low `average_position` values and low `total_points` values. They have also a low participation. These datapoints aren't present in the scatterplot on the right hand side. This seems to indicate that there are many cyclists that get good results in few stages, and then end their career
- To low `total_points` values, there corresponds many values for `average_position`. As the Total Points increase, the Average Position slowly decreases, as one would expect. We observed a weak negative correlation between the `avg_points_per_race` and `average_position` features
- There's no clear trend with respect to the number of attended races

There are very few (21) cyclists that have accumulated a huge amount of points (over 7500). Their `average_position` value is usually below the population mean (which is 76), but not always. Some of them attended a lot of races, some of them "just" 150.

2.2.5 Nationality of the cyclists and of the competitions

More than 85% of the cyclists in the dataset are europeans, and all but four competitions in the dataset are held in Europe (the exceptions are in Canada and in the UAE, but they're heavily underrepresented in the dataset, and are recent).

All the data concerning the number of hosted competitions (thus not counting the individual stages) per country, and the nationalities of their winners⁴ is summarized in figure 3 on the next page. It turns out that even among the european nations there's an uneven representation when it comes to winning and hosting races.

In figure 3 we can see the number of victories of a given country (x axis) in competitions hosted in a given country (y axis). It is clear that competitions tend to be won by cyclists of the same nation in which it's ran. By summing along the rows we can read how many competitions each country hosted (the top one is Belgium, with 300), and read how many victories each country scored overall (again, Belgium has the most, 252).

We remind that the leaderboard considered here is a minor leaderboard that can be very different from the main one, and we stress the fact that cyclism isn't a sport with country-based teams. Statistically this is an interesting phenomenon nevertheless.

2.2.6 The `victories_by_points` feature

Only 363 cyclists (5.92%) have a positive value for this feature. They're outliers also with respect to the `avg_points_per_race` feature, with most of them having values greater than 20, though the range is wide. As one would expect, all of them have `average_position` values below average.

⁴Intended as victories by points as described earlier

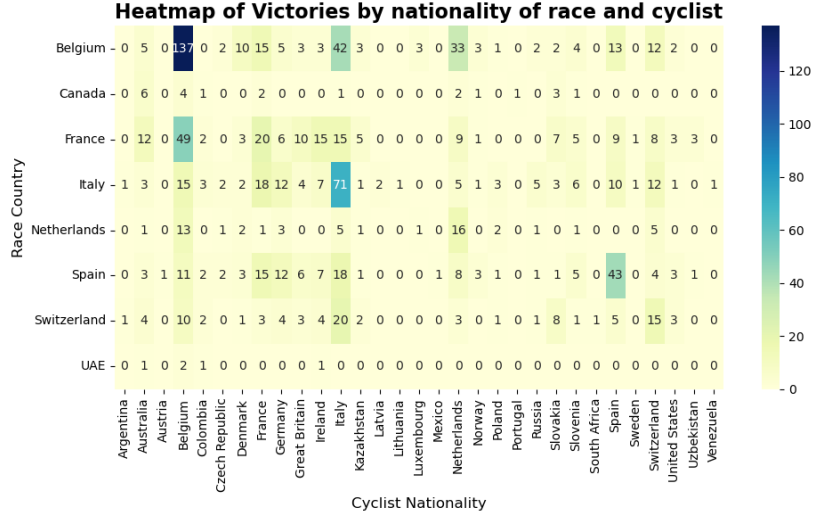


Figure 3: *Pivot table with the competitions-hosting countries and the competitions-winning countries*

2.3 Closing remarks

For all features considered in this section, there's a deeper study with more plots, more numbers and more comment in the `data_understanding_transformation` or in the `data_exploration` notebooks.

This report couldn't possibly contain all the investigations that have been performed. Furthermore, there's always something more to explore, there are always more features to engineer and study. The ways to improve what's been done are countless.

3 Clustering

We tried different techniques; Prototype based, Density based and Hierarchical, on almost the same set of features, differentiating them where we thought they might offer some insights or result in saner clusterings. In the following sections, our findings will be reported.

3.1 Prototype Clustering

The algorithm used in this part is K-Means. To decide the number of clusters, silhouette score and SSE were considered since there was not a clear candidate number when using the elbow method. This is probably due to the fact that it seemed like there were only one or two big clusters with some noisy points in both the dataset. When choosing the number of clusters, we also decided the criteria with which to pick the starting points evaluating the two most used initialization techniques: `random` and `k-means++`, without finding much difference in the resulting clusters.

3.1.1 Cyclists

The work on the cyclist dataset hasn't added too much knowledge from what we already discovered during the data exploration. In order to apply k-means++, we removed some features that had high correlation to other, and some other that were categorical, restricting the algorithm to work with just 6 features (**BMI**, **race_count**, **total_points**, **avg_speed_cyclist**, **cyclist_age**, **mean_stamina_index**). The cyclists were grouped in 5 clusters (see figure 4a) to which we were able to assign a deeper meaning that just proximity in a 6 dimensional space. The most interesting clusters are cluster 2 (green) and cluster 0 (blue): the former is composed by just a bunch of cyclist but they are "the strongest", while the latter is the biggest cluster and his elements are quite heterogeneous but it seem that they are "the average cyclists" that have low speed relatively to the rest of the dataset.

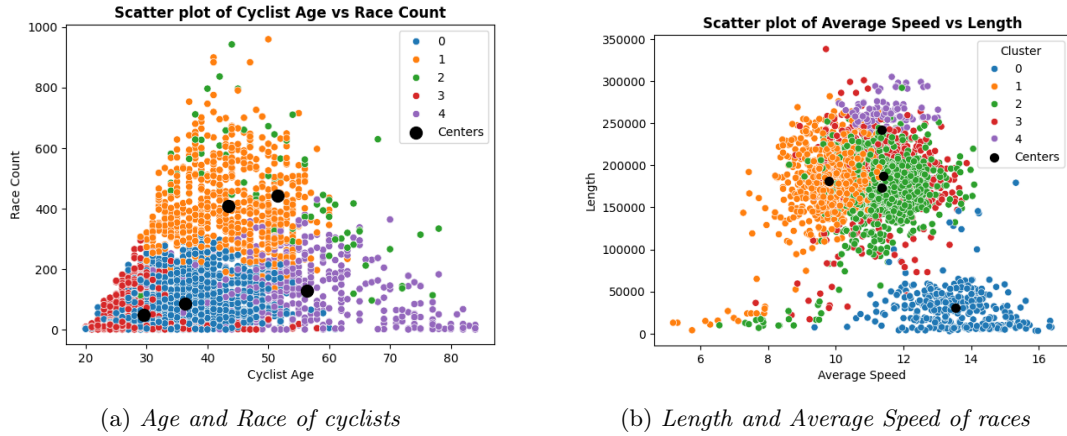


Figure 4: 2D projection of K-Means cluster over the 2 separate dataset

3.1.2 Races

The second application of K-Means 4b yielded some more interesting results that allowed us to have a better insight into the races dataset. As for the other dataset, we did not run the algorithm over all features but we used a subset initially composed of: **points**, **length**, **climb_total**, **startlist_quality**, **average_speed**. Here we decided to create 5 clusters (always based on the elbow method). The most visible and well separated cluster was one containing only races with higher points (cluster 4), the second clear group separated from other was a group made mostly of short races (cluster 0); this two clusters were also very homogeneous. This is quite simply explained for the cluster of short race: they are short, so cyclists go faster and it is hard for the climb total to be a high value (indeed it is the cluster with the lower climb total on average). Looking closer at the cluster containing top races, we discover that those races are all long races and the average speeds are all in a small range.

In the analysis of this clusters we went a little deeper than we had done with the cyclists dataset. We tried to see if the cluster had some relation with the feature that we did not considered, and we found some interesting stuff. First of all we've discovered that almost all races of cluster 3 are french (more in particular 500 out of 513 are stages of 'le Tour de France'); this is caused by an over representation in the dataset of that particular race. Representing the distribution of the profiles over the clusters we found that some cluster were to some extent representing races with lower or higher profile, and this make sense since profile describes how difficult a race is based on the length and the total climb; this patter was even more noticeable when we executed the algorithm removing the `points` feature. A comparison of the clusters with `is_staged` and `is_tarmac` revealed that the top races are not divided in stages and are not competed on tarmac (probably on multiple terrains given their length). This last discovery could have been found also during the data analysis but it was made noticeable by the clustering.

3.2 Density-based Clustering

The algorithm chosen for the density-based clustering was DBSCAN. The clustering was run separately on the cyclists and on the races. A full discussion, with all the visualizations, can be found in the `density_based` notebook.

3.2.1 Cyclist density-based clustering

The features value have been standardized, and only the euclidean distance has been used.

Only the `birth_year`, `height`, `race_count`, `total_points`, `average_position`, `avg_speed_cyclist`, `mean_stamina_index` and `victories_by_points` features have been kept. Datapoints with missing values have been removed, so the clusterings involved 2971 cyclists out of 6134.

Multiple attempts were made, gradually adding in more and more complexity: a smaller search range of the `min_samples` hyperparameter once its low relevance with respect to the obtained clustering had been established, smaller values for the `eps` hyperparameter. For the clusterings involving the cyclists, the `eps` value has been picked by hand for every value of `min_samples`, using the elbow method on the k -th distances plot. On a next attempt, the `victories_by_points` feature was removed, given that its distribution was extremely skewed (as most cyclists have a value 0 for this feature), and the `mean_stamina_index` range was reduced to remove (very few) outliers.

Despite all of this, all clusterings yielded one big cluster. All clusterings were evaluated using the Silhouette score; for all attempts at least one clustering was evaluated using a trivial internal measure, i.e. summarize features' values of the noise points.

When multiple clusters were found, the smallest ones had extremely few datapoints, and it turned out that these clusters were "soon to become noise points"; i.e. with a different but close value of an hyperparameter, the points in that cluster would become noise points. Basically the clustering algorithm was behaving like an outlier detector.

This is why on our last attempt we decided to compare the clustering with a binary label denoting whether the datapoint had a positive value for the `victories_by_points` feature. Using this as the true label (266 positive samples, 2705 negative samples) and the clustering as prediction,⁵ a precision of 0.347 and a recall of 0.406 were reached. Such bad results indicate that the other features are not a good predictor of the `victories_by_points` feature if one wishes to simply consider the density, and a more sophisticated approach would be needed.

3.2.2 Races density-based clustering

Only the `length`, `climb_total`, `startlist_quality` and `average_speed` features have been used. We removed the datapoints with missing values (only one the `climb_total` feature had them), and ended up with 385806 samples. After removing the duplicated with respect of the race URL (in fact these features relate to the race itself, so in the dataset these values are replicated for each participant), we were left with 2727 datapoints. As with the cyclists, multiple attempts were made, changing things gradually.

On a first attempt, very few `min_samples` values were considered, but for each there were seven possibilities for `eps`. The results of the grid search are visible in figure 5a. From it, considerations of the impact of the hyperparameters can be drawn; for example it can be seen how increasing ϵ puts the datapoints in the main cluster. One clustering was studied in more detail, and it turned out that - besides the main cluster - a cluster of short races ran at lower speed was found (mostly ITT, by comparing with the data exploration).

On a second attempt we tried to use the Mahalanobis distance, the idea being that the correlation between the data was such that it assumed a more elongated shape along some axis, and a more contracted shape along others; thus using the same scale to measure the distance in all directions gave less accurate results. Using the Mahalanobis distance tantamounts to correcting these distortions.

The clusterings obtained weren't that different in terms of external measures such as Silhouette score, number of noise points, but a more interesting clustering than before was obtained. The pairplot with the outcome is in figure 5b. By comparing the `length` vs `average_speed` plot in the top right corner of the figure with the corresponding scatterplot in the exploration notebook, we can say right away that the points in Cluster 1 have low `profile` value and mostly correspond to ITT stages.

This last fact made us wonder about the ability of the clusterings produced by DBSCAN to predict the ITT label. For this we used a confusion matrix approach similar to the one done previously, using the belonging to Cluster 1 as predicted label and being an ITT race as true label. The clustering achieved a recall of 0.811 - indicating that most ITT datapoints had been found - and a precision of 0.823, meaning that the clustering mostly included ITT races. These results together tell us that the information about

⁵The chosen clustering has `eps` = 1.4, `min_samples` = 24. It was chosen because it yielded a number of noise points close to the number of positive samples according to the defined label. We were not actually trying to find the best clustering of the data in this attempt, but just to assess the "goodness" of a clustering as a detector of a feature value.

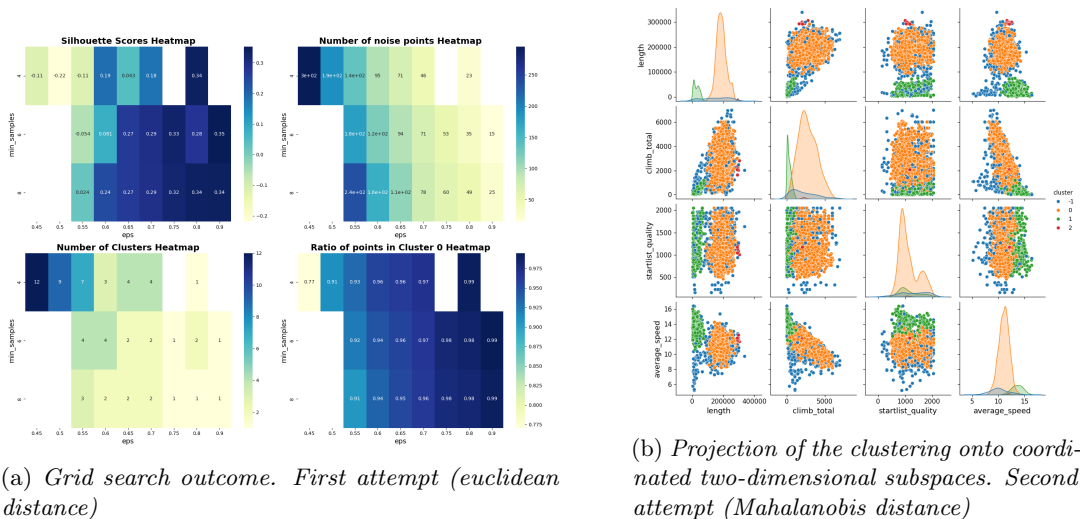


Figure 5: Grid search (left) and pairplot (right) for the DBSCAN clustering algorithm on the races

being an ITT race or not is really easy to obtain just by using the spatial location of the data. The information provided by the `stage_type` feature is easily obtainable from the other features.

The third attempt at clustering the races using DBSCAN involved scaling the data using the `MinMaxScaler` instead of standardization, the idea being that since the datapoints are all tightly packed together and rescaling the distances seemed to be helpful, it was worth trying what would happen if a predefined range was given (i.e. try with a different linear transformation of the features than the one used by the Mahalanobis distance). Since this scaling puts all datapoints in a "box" (i.e. a ball for the l_1 distance), we also tried using the Manhattan distance. The results - visible in the notebook - were underwhelming.

To summarize, density based clustering on the data was unsuccessful. We conjecture that this is because the driving assumption that the datapoints are grouped in the space into "zones" of higher density separated by zones of lower density doesn't hold. Let alone that they do so in a semantically meaningful way. The visualizations of the data projections on 2D subspaces support this claim, but a future improvement of the project could be to try to understand this with advanced embeddings such as t -SNE.

3.3 Hierarchical Clustering

Because of its high resemblance to a categorical variable, we decided to not cluster the `points` feature to weed out bad clusters. Since `steepness` defines a relation between `length` and `climb_total`, we thought that hierarchical clustering could benefit from this feature because it is correlated with `profile`, which is categorical, and is a continuous variable with a distribution that has no sensational outliers, thus not impairing the clustering algorithms. We decided to go with agglomerative clustering for its ease of

use and the possible search over different linkage types (or methods to analyze cluster similarity) to analyze.

After scaling the data, we computed the silhouette score for different combinations. Because of its sensibility to outliers and the great presence of them in the dataset, we decided to keep the number of **clusters** over 2 in our search. We also decided to keep it under 11 to have meaningful sets of data to analyze instead of many small clusters. As for the **linkages**, we decided to include MIN, MAX, Group average and Ward's method. We chose to analyze the results that had the best silhouette scores and meaningful plots. We mapped the most interesting ones into a three dimensional feature space using UMAP. We also computed the dendograms for the illustrated methods used to cluster the cyclists. They were used as another measure to check the validity of the clusterings produced. (check the `hierarchical_based` notebook for these two features).

3.3.1 Cyclists

To cluster the cyclists, we used the following features: `bmi`, `race_count`, `total_points`, `avg_position`, `avg_speed_cyclist`, `cyclist_age`. The dendograms for every method can be seen in the notebook. On MIN and Group Average, we won't be saying much. They identified a giant globular cluster and another two clusters of outliers. Group Average was a bit more robust, fitting more data in the second cluster but keeping its size under 20.

MAX, as expected, did much better. Its maximum silhouette score, 0.39, was reached with three clusters. It appears very small compared to MIN (0.77) and Group Average (0.46), but its plot was the only meaningful one that didn't have clusters containing a single datapoint (albeit Average had "just" one cluster composed of an outlier, compared to MIN). We analyzed the mean for each cluster, alongside their size. The first cluster represents the bulk of amateur cyclists, mostly young, that did not reach impressive average positions and did not accumulate an high number of total points. The second cluster represents a small group of competitive cyclists that perform exceedingly well and have the highest experience. It's basically a cluster of outliers. The third cluster consists of cyclists that managed to distinguish themselves from the amateurs. Considering the mean age of this cluster, we can say that most of the cyclists belonging to this cluster are "vintage" cyclists. The most satisfying cluster was obtained with Ward's Method. Its best silhouette score was reached with three clusters (0.23). The algorithm was less influenced by the outliers. According to the mean values for each cluster, it followed the same approach as MAX, getting more cyclists out of the "mediocre" cluster and filling the other two, especially the "elite" one. Since it's the clustering of the cyclists we found to be more satisfying, we decided to plot it on a reduced feature space with UMAP (can be seen in the relative notebook).

3.3.2 Races

For the races we used the following features: `length`, `climb_total`, `startlist_quality`, `average_speed`, `steepness`. The most satisfying clustering were those achieved with Group Average, both with three and four clusters. Three clusters reached the best

silhouette score (0.435). We analyzed their means: the first gathers shorter, steeper races that proved to be more challenging, considering the very slow speed. The size hints that such difficult races only compose a small subset of all the races. The second cluster denotes all the longer races, which are more endurance-focused. An higher average speed with respect to the first cluster was expected as the steepness dramatically falls, indicating less aggressive, longer climbs. The startlist quality is the smallest, indicating with the fact that the cluster size is so high, that these races are more common and faced by more people with different skill levels. The third cluster indicates races that are on almost flat terrain, with the highest average speeds. These sprint-races present a bit of a higher startlist quality that indicates they are dwelled by slightly more professional individuals. Since this clustering proved to be satisfactory, we decided to plot it in a reduced feature space with UMAP.

Group Average with four clusters reached a score of 0.41. The means of the clusters can be seen in table 2. In this instance we can see how the previous iteration decided to aggregate data into the first cluster. We can see that the `climb_total` feature does not decrease monotonically between clusters ordered by length.

Cluster	Length	Climb Tot	Startlist Qual	Avg Speed	Steepness	Size
0	199452.30	3293.46	1114.73	10.73	1.72	14611
1	32429.94	1300.23	1381.85	9.04	3.99	339
2	33113.64	362.76	1180.46	13.12	1.12	3328
3	15721.99	999.33	960.86	6.93	6.50	312

Table 2: Mean values of the features for every cluster obtained from Group Average

3.4 Observations and comparisons

Both density and hierarchical based techniques were greatly hindered by the presence of outliers, computing clusters solely composed of them. Unfortunately, density based techniques did not prove to be good predictors of outliers with respect to one feature. Both K-means and Density based techniques found short races with above average speeds, almost all ITT, basically finding the same cluster. Hierarchical techniques, on the other hand, distinguished short races that are "slow" and "fast".

4 Classification and Explainability

These tasks were merged into a single section because the findings we produced are deeply intertwined. We mainly explored the behavior of XGBoost [3] and Neural Networks on the augmented dataset, joining cyclists' and races' data. XGBoost has been studied with SHapley Additive exPlanations (SHAP) [5, 6] and DICE [7, 8], and it has had a globally interpretable model extrapolated from it [4]. For the Neural Networks we opted to use SHAP.

Relevant files to these tasks can be found in `/Classification/DecisionTree.ipynb`, `/Classification/neural_network.ipynb` and `/Classification/XGBoost.ipynb`. The utility functions to load the dataset and recompute various features of the dataset that are intrinsically dependent on the date are found in `/utility/classification_utility.py`. For example, the total points earned by a cyclist should account for all the races they participated in up to the race being considered.

4.1 Validation strategy

For the validation strategy, we had to choose between two basic assumptions. If the distribution of the merged dataset is independent and identically distributed or if it changes over time, based on the `date` feature.

If the first assumption was correct, we would have randomly sampled a validation set equal to 20% of the training set and implemented a K-Fold Cross Validation.

Under the assumption that the distribution changes over time, we thought that keeping a validation set of all the data after 2021 and before 2022 would have made our models more resilient to the possible trend shifts introduced by the test set. Even if some features already incorporate results from cyclists' preceding performances, we thought that ensuring that the model generalized well over 2021 was pivotal for training. This meant, though, resorting to a singular hold-out validation. Since hold out validations are usually faster than K-folds, we had more computational resources to extend our grid search over the possible hyperparameter space. The final data split is then described in table 3.

Set	Data %	Earliest date	Latest date
TR	82.77%	28/02/1970	01/01/2019
VL	10.14%	01/01/2019	31/12/2020
TS	7.09%	31/12/2020	29/07/2023

Table 3: *Training, validation and test set split*

4.2 Metrics

The percentage of positive samples of TR and VL is $\approx 14\%$ for both, which is fairly low. As for how the task and the dataset are structured, the number of cyclists reaching the top-20 podium is heavily underrepresented.

We thought that our model should prioritize giving good predictions for the cyclists that will enter in the top 20; while still keeping a high accuracy. We obviously wanted also a good performance on the negative class in order to reduce the false negatives. For these reasons we opted to make our model maximize the macro average F_1 score, giving a good balance to the trade-off between the precision and recall of both classes.

4.3 Decision Tree

This is the simplest model that was tried; the objective of creating a decision tree model was not to study its behaviour, but to compare it with a forest based tree (FBT) created starting from an XGBoost model⁶. This is the reason why we didn't go further than one grid search in our hyper parameter tuning. We used a subset of features (the ones used for the FBT) that is composed by: `total_points`, `avg_points_per_race`, `average_position`, `avg_speed_cyclist`, `race_count`, `profile`, `startlist_quality`, `cyclist_age`, `steepness`, `participants_number`, `bmi`, `average_position_var`.

The hyper parameter space of the grid search is described by 4; some were added just to study the model's behaviour. For example, if the best `max_depth` resulting from the grid search had been 20, we wouldn't have considered it for comparison with the explainable model induced from the forest.

Hyperparameter	Values	Selected
Max depth	{5, 10, 15, 20}	5
Min samples split	{1, 2, 3, 4}	2
Min samples leafs	{1, 2, 4, 8, 16, 32}	1
Criterion	{gini, entropy}	entropy
Min weight	{1, 2, 4, 6}	4

Table 4: *Hyperparameter grid search values. (Min weight is short for weight of the minority class)*

The hyperparameters of the model chosen to be compared with the FBT are listed in the rightmost column of table 4. This model reached an F_1 score of 0.65 on the validation set and an accuracy of 0.78. Note that this is not the best tree that came out of the grid search, but we wanted a tree with reasonably low depth in order to have a fair comparison. In any case the best model's F_1 -macro was not much higher (0.66) with a depth of 10.

4.4 Decision Forests

Dataset

We executed various trials, with different sets of features, obtaining almost always the same results. The ones used to train the best XGBoost model were: `total_points`, `avg_points_per_race`, `average_position`, `avg_speed_cyclist`, `mean_stamina_index`, `race_count`, `length`, `climb_total`, `profile`, `startlist_quality`, `cyclist_age`, `steepness`, `is_tarmac`, `height`, `weight`, `bmi`, `home_game`⁷. After seeing the high importance assigned to `average_position` by the SHAP values for both XGB and the neural network, we decided to add 2 more features: the variance of the average position (`average_position_var`) and `participants_number`; hoping to help the models to infer the probability of being in

⁶The algorithm is explained in depth in the paper we presented during the Data Mining course [4]

⁷A binary feature which denotes whether the cyclist and the race are from the same country

the top 20. Since from the beginning we had in mind to create a FBT with a particular library (the one used in [4]) we couldn't use the `season` feature because categorical values were not handled by the code. Boolean columns had to be preprocessed, to make them integer that either take values 0 or 1.

4.4.1 Model selection

After this we executed a grid search to study which hyper parameters were more important to tune in order to have better results. It turned out that the most influential one was the `max_depth` of the tree in the forest, while other hyper parameters did not impact the F_1 score as much. After this, we tried to fix various `max_depth` values and execute a random search for each one, but weren't able to improve the first result by a significantly. We also tried to decrease the oversampling of the minority class but obtaining always result that were around 0.67/0.68 of F_1 -macro. Since we judged these values of F_1 unsatisfactory, we tried to see if the model was less performant on elder races. The decaying of the performance over time was maybe present, but for sure it wasn't the model's main problem.

The model's hyper parameter space and the best ones resulting from the various parameter tuning are shown in 5. The best model reached a score of almost 0.68 F_1 -macro.

Hyperparameter	Ranges	Best
N boost round	{100, 800}	500
Learning rate	{0.05, 0.2}	0.05
Lambda reg	{0.1, 5}	5.0
Early stopping	{10, none}	none
Max Depth	{3, 15}	12

Table 5: *Hyperparameter grid search values. We also tried various percentage of oversampling, from not adding any to a perfectly balanced dataset; the best option turned out to be making the minority class present at least half of the time of the majority one*

4.4.2 Explainability

The first explainability technique used was SHAP [6], which provides insights into the features the model considered most important. The most influential features identified were: `average_position` (lower values increased the probability of class `True`), `avg_points_per_race` (higher values increased the probability of class `True`), and the age of a cyclist at a given race (higher values decreased the probability of class `True`). To validate these findings, we attempted to sort cyclists in races based on a single attribute at a time. Interestingly, using only `avg_points_per_race` yielded better results than using just average position. However, a deeper analysis of SHAP values revealed that average position primarily impacts the model's decisions for extreme values, `avg_points_per_race` consistently influences the classification.

This analysis also uncovered broader patterns in the dataset. Firstly, racing in one’s native country appears to give cyclists a slight advantage, though it is not a major factor in determining races outcomes. Secondly, the BMI of winners shows some variation based on race profiles, with higher BMI cyclists tending to win more frequently in races with profile 1.

After this we also tried DICE [8]. The counterfactual analysis using DiCE highlighted an obvious yet important finding: cyclists are more likely to lose if they are slower. While this result is expected, it aligns with the insights derived from SHAP, providing validation from two different explainability techniques. Additionally, the analysis confirms that the model relies on intuitive features a human would consider when making similar judgments. For example, changing a cyclist average speed from 11.0m/s to just 2.6m/s (9.36km/h) changes the classification from positive (winner) to negative (not in the top 20).

However, the analysis also revealed some strange behavior. In one sample the `avg_speed_cyclist` was increased from 11 m/s to 13 m/s, the model unexpectedly changed the classification from winner to loser. This is likely due to overfitting on specific data points and represents a flaw in the model’s behavior.

The last thing we did for explaining the XGBoost model was try to create a tree induced on our model as in [4]. While this reduced the original model’s F_1 -macro score to 61, it improved interpretability, as expected. However, the fidelity (matching predictions) was 71%, which is lower than the original paper’s but understandable given our use of a small subset (400 samples) due to the algorithm’s computational cost. We believe that this results could likely improve by using more sample.

If we compare the FBT to the classic decision tree, we must conclude that - in our scenario - the latter is the preferred choice due to its ease of use and, more importantly, its better performance. Our divergence from the results of the original paper can likely be explained by two factors: the use of a small fraction of the data and the difference in target metrics. The original paper evaluated accuracy and AUC, two metrics that their method optimize by design.

4.5 Neural Networks

Dataset

For this task we considered all the features used for the XGBoost model (see section 4.4.1), with the addition of `stage_type`, `season`.

To respect the rules of the task, some features had their values recomputed considering only the data available before the `date` indicated in their respective row.

We imputed some values to prosecute the training of the neural networks without dropping half of the rows. Where the values were absent we imputed `height`, `weight` and `cyclist_age` for each cyclists using a simple mean of the dataset. This allowed us to infer the missing `bmi` values as well. The `climb_total` has been computed as 5% (which is the imputed `steepness`) of the `length` of the races. Randomly imputing that with a mean would have created inconsistent data. We imputed `profile` by training a Random Forest Classifier using `length`, `climb_total`, `steepness`, `average_speed`, and `stage_type` as

its input features. The precision on a hold-out randomly picked test set had an accuracy of ≈ 0.99 .

`season` had been used as a one dimensional feature since one-hot encoding didn't give any perceptible result.

Before proceeding to the definition of the model, we standardized every feature.

4.5.1 Model Selection

We incrementally checked new features to empirically test whether each one would have brought any sensible advantage without performing a grid search that was too time expensive.

We applied oversampling to reduce the difference in representation between the minority and majority classes. We used both Random OverSampling and SMOTE, reaching the same results with them. This boosted the recall of the model (which started at abysmal values) to very good amounts, proportional to the ratio between the classes, nearing 80%. Unfortunately, an higher ratio in oversampling meant a degrading in accuracy. Dropout proved to be a good form of regularization to prevent the model from overfitting to the training minority samples. Thus, we balanced the effects of oversampling, reducing the impact on precision, but losing the great values in recall. A good balance was found in keeping the ratio of the majority and minority classes at 0.35 and searching between a Dropout rate of 0.2 and 0.5. We also decided to make use of an early stopping criteria based on the recall of the validation set, which is the metric that most hindered the F_1 score. The grid search was made on the parameters illustrated in table 6.

Hyperparameter	Values
Batch Size	{16, 32, 64}
Number of Layers	{1, 2, 3, 4}
Number of units	{32, 64, 128}
Activation	{Tanh, ReLU}
Dropout Rate	{0.2, 0.5}
Optimizer	{Adam, SGD, Lion}
Epochs	{5, 10}

Table 6: *Hyperparameter grid search values for the Neural Network.*

After this grid search, more fine grained ones followed, noticing that there were small improvements when using more layers and units. Reaching the fifth layer of depth, we used Batch Normalization to mitigate the covariance shift. The model we ended up selecting from this search is shown in table 7. It reached a macro average F_1 -score of 0.69, greatly hindered by the inability to bring both precision and recall of the True class over 50%.

Batch	Layers	Units	Activation	Dropout	Optim.	Epochs	Oversampl.
32	5	256	ReLU	0.2	Adam	10	0.35

Table 7: Model selected from the fine-grained grid searches

4.5.2 Explainability

To explain the model, we used SHAP. To do so, we randomly selected 50 datapoints (from the validation set) to feed to the explainer as a baseline to get the Shapley values, for 100 predictions of the validation set. For all these predictions, we produced a force plot and a summary plot (both can be found in the `neural_network` notebook) that show the contribution of each feature. This gave us some insights about the model’s behavior.

`average_position`, `cyclist_age`, `profile` and `average_position_var` (which had been added later): different values of these features strongly influence predictions. As we expected, statistics that give an insight on the cyclists’ previous positions are used a lot by our model. As it seems, an higher age implies less chances to win a race. This can be seen as cyclists having more chances to get good positions when they are in their "prime" age. Considering the fact that the increase or decrease in SHAP values of `profile` is not monotonic, one could say that there are a lot of cyclists more inclined to win certain races than others.

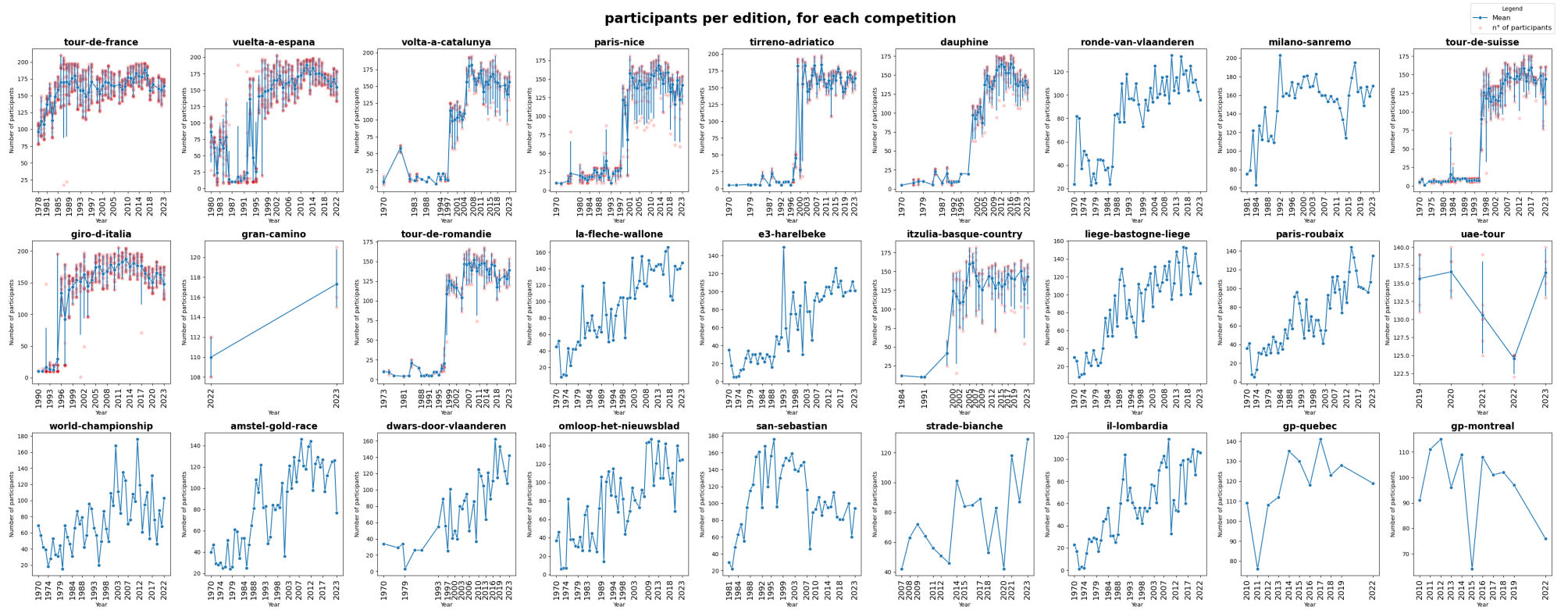
`avg_points_per_race` and `total_points`: these features also play a critical role but have slightly less variance in their SHAP values compared to the top features. These general statistics about how good a cyclist performs at a given time help the model infer its possible position, as we expected.

`avg_speed_cyclist`: This feature has very skewed values for the lowest values. Intuitively, it’s difficult to get on the podium if you are a slow runner.

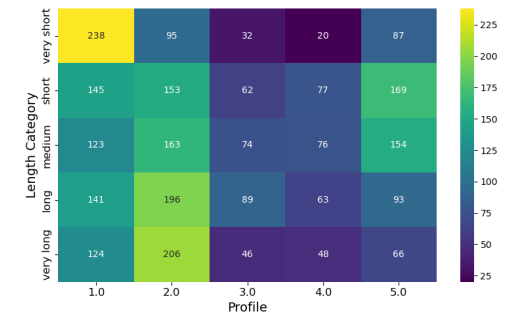
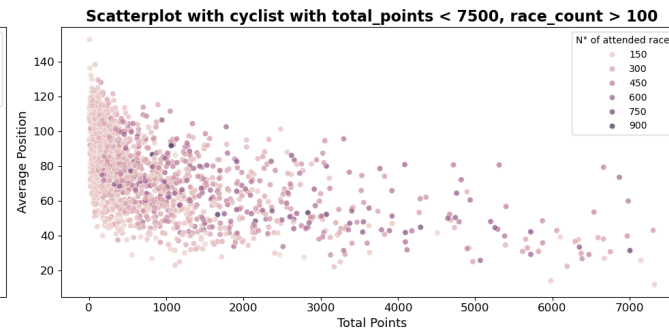
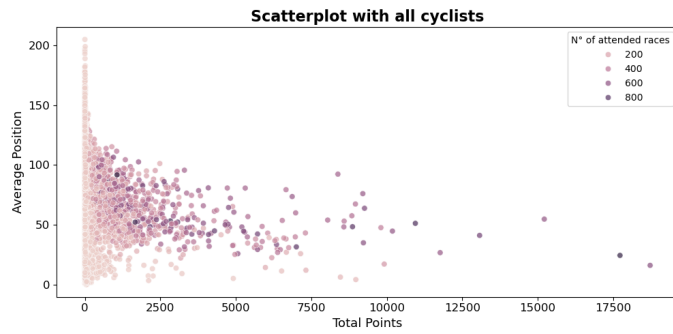
`race_count`, `startlist_quality`, `mean_stamina_index` and `bmi`: These have limited impact compared to other features but still contribute marginally. For our model, more "massive" cyclists have a slightly harder time winning. An honorable mention goes to the `season` feature, which somehow managed to influence the model outcome. The model inferred that under certain seasons, some cyclists with particular features might perform slightly better than others.

4.6 Performance on Test Set

In the end, the model that maximized the F_1 -score was the Neural Network we picked before. It’s been retrained on the sum of both the training and validation sets, bumping the epochs to 30 while still using a loss based stopping criteria, giving it 5 epochs of patience. In the end, the F_1 score on the test set was 0.7, which means our model managed to not overfit without a validation set and to generalize the future distributions of 2022 without losing performance. Instead, the performances were better. We think this is due to the fact that data from 2022 is much cleaner and has far less outliers than past data. Having less noise to predict, the model worked better.



(a) Participation to the 27 unique competitions in the dataset, over the years. The red dots represent the number of cyclists in the stage. All together they represent a participation distribution for that edition of the competition. The blue dot is its average, and the vertical error bars represent the interval between the 5th and 95th percentile of such distribution.



(b) Scatterplots of **total_points** vs **average_position**, colored by number of attended races

(c) Heatmap showing how many races per length category are in each given profile

Figure 6: Participation trend to the competitions in the dataset (top), an example of scatterplot (bottom left) and heatmap (bottom right) produced in the data understanding phase.

5 Conclusions

In this report we’ve summarized the work that we did for the Data Mining exam’s final project. We’ve described the activity that was done for the Data Understanding and Feature Engineering parts, the Clustering part, and the Classification and Explainability part.

We tried to be as precise as possible, trying to give the least possible things for granted. This notwithstanding, often we had to redirect the reader towards paragraphs, plots and analysis that are present in our notebooks for the details; after simply referring or describing what could be found in there. This was necessary, as this report isn’t enough to fully grasp the work done for this project. This being said, we make no claim of exhaustiveness; in fact we’re aware that there are both limitations - some of which have been discussed in this document - and further avenues for improvement in our work.

All in all, the project has been a challenge that taught us a lot not only about Data Mining, but also about working and coordinating as a team.

References

- [1] *ProCyclingStats.com | Cycling statistics, results and rankings*, <https://www.procyclingstats.com/>.
- [2] M. Madzin (2024), *procyclingstats 0.2.1*, GitHub Repository, available at: <https://pypi.org/project/procyclingstats/>.
- [3] T. Chen and C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–94, 2016. <https://doi.org/10.1145/2939672.2939785>.
- [4] O. Sagi and L. Rokach. *Approximating XGBoost with an Interpretable Decision Tree*. Information Sciences 572 (September 2021): 522–42. <https://doi.org/10.1016/j.ins.2021.05.055>.
- [5] S. Lundberg and S.I. Lee. *A Unified Approach to Interpreting Model Predictions*. arXiv, November 25, 2017. <https://doi.org/10.48550/arXiv.1705.07874>.
- [6] *SHAP documentation* <https://shap.readthedocs.io/en/latest/>.
- [7] R.K. Mothilal, A. Sharma, and C. Tan. *Explaining Machine Learning Classifiers through Diverse Counterfactual Explanations*. In Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, 607–17, 2020. <https://doi.org/10.1145/3351095.3372850>.
- [8] *DICE documentation* https://interpret.ml/DiCE/dice_ml.html.